LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 06 – INHERITANCE



Jury Maxwell

103132400023

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance (pewarisan sifat) merupakan salah satu pilar fundamental dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan suatu kelas (subclass atau derived class) untuk mewarisi seluruh atribut, field, dan method yang dimiliki oleh kelas lain yang lebih umum (superclass atau base class). Melalui mekanisme pewarisan ini, arsitektur perangkat lunak dapat mengeliminasi duplikasi kode (code redundancy), meningkatkan usabilitas kembali (reusability), serta membangun relasi hierarkis 'is-a' (adalah bagian dari) antar objek secara terstruktur dan terstandarisasi. Pada bahasa pemrograman Java, inheritance direalisasikan menggunakan kata kunci extends, di mana seluruh karakteristik non-private superclass otomatis diturunkan ke subclass.

1. **Konsep Superclass dan Subclass:** Superclass merupakan kelas induk yang mendefinisikan state dan behaviour umum, sedangkan subclass merupakan kelas turunan yang memperluas atau mengkhususkan perilaku kelas induk. Karakteristik umum pada superclass akan dimiliki juga oleh subclass-nya, sementara subclass dapat menambahkan atribut maupun method baru yang bersifat spesifik.
2. **Penggunaan Keyword super:** Kata kunci super digunakan di dalam subclass untuk mereferensikan anggota dari superclass secara eksplisit. Keyword ini memiliki dua fungsi utama: memanggil constructor milik superclass (constructor chaining) melalui sintaks super(argumen) yang wajib diletakkan pada baris pertama constructor subclass, serta mengakses method atau variabel superclass yang tertimpa atau tertutup (shadowed) oleh anggota subclass.
3. **Method Overloading:** Kemampuan sebuah kelas untuk mendefinisikan dua atau lebih method dengan nama yang identik namun memiliki parameter yang berbeda (baik dalam jumlah argumen, tipe data, maupun urutan tipe datanya). Overloading memungkinkan kelas menyediakan berbagai antarmuka fungsionalitas yang fleksibel tanpa harus mendefinisikan nama method yang berbeda-beda untuk tindakan yang serupa.
4. **Method Overriding:** Mekanisme di mana subclass mendefinisikan ulang method yang telah ada pada superclass dengan nama, return type, dan daftar parameter (signature) yang sama persis. Ketika instance dari subclass memanggil method tersebut, implementasi spesifik milik subclass yang akan dieksekusi, bukan lagi implementasi milik superclass-nya. Jika subclass masih ingin mengeksekusi logika kelas induk, pemanggilan dapat dilakukan dengan memanfaatkan super.namaMethod().

B. Guided

1. Inheritance Dasar (Superclass dan Subclass)

File: A.java

```
package guided.gd1;

public class A {
    int x;
```

```
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

File: B.java

```
package guided.gd1;

public class B extends A {
    int z;

    void jumlahXY(){
        System.out.println("Jumlah = "+ (x+y+z));
    }

}
```

File: Demo.java

```
package guided.gd1;

public class Demo {
    public static void main(String[] args) {
        A superclass= new A();
        B subclass = new B();

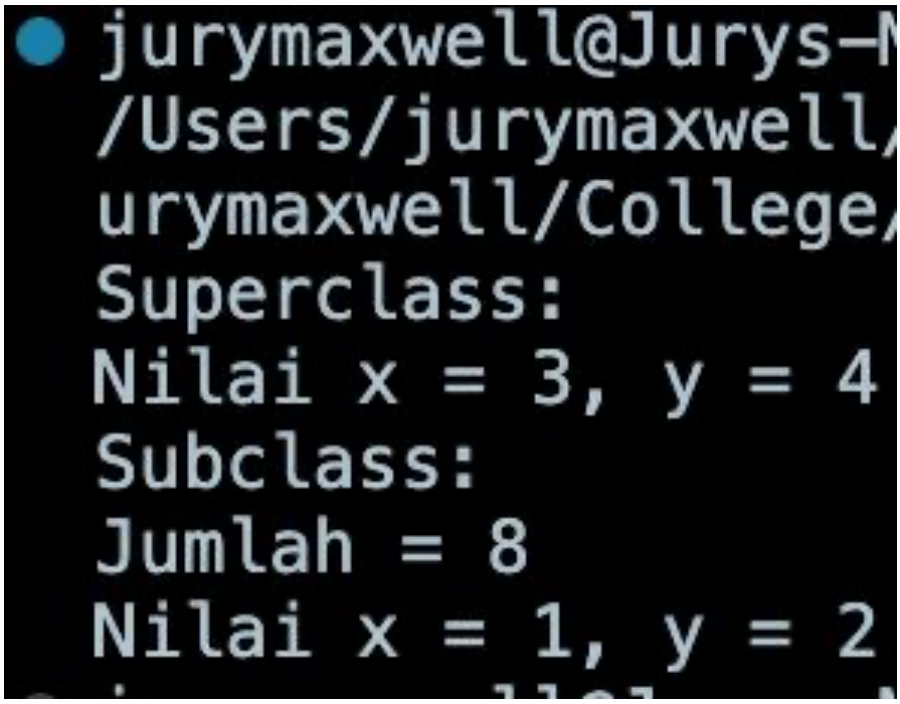
        superclass.x=3;
        superclass.y=4;


        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x=1;
        subclass.y=2;
        subclass.z=5;

        System.out.println("Subclass: ");
        subclass.jumlahXY();
        subclass.tampilXY();
    }
}
```

Output :



2. **Penggunaan Keyword super dan Constructor Chaining**
   File: Employee.java

```java
package guided.gd2;

public class Employee {
    private String name;
    String departemen;

    public Employee(String name){
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("Name: " + name);
    }
}
```

File: Manager.java

```java
package guided.gd2;

public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
```

```
    }

    public void tamplInformasi(){
        super.tampilNama();
        System.out.println("Alamat: " + alamat);
        System.out.println("Departemen: " + super.departemen);
    }
}
```

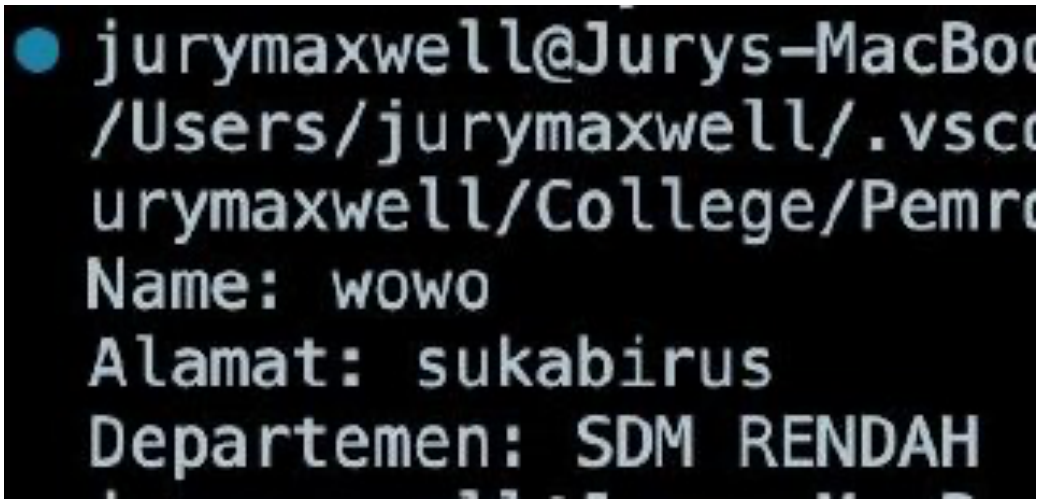
File: Demo.java

```
package guided.gd2;

public class Demo {

    public static void main(String[] args) {
        Manager wowo = new Manager("wowo", "sukabirus");
        wowo.departemen = "SDM RENDAH";
        wowo.tamplInformasi();
    }
}
```

Output :



3. **Method Overloading pada Class Lagu**

File: Lagu.java

```
package guided.gd3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
```

```
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1,String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " +
pencipta);
    }
}
```

File: Main.java

```
package guided.gd3;

public class Main {
    public static void main(String[] args) {
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
jurymaxwell@Jurys-MacBook-Air modul-06-ir
s/jurymaxwell/.vscode/extensions/redhat.j
/College/Pemrograman\ Berbasis\ Objek/Pra
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku
```

4. **Method Overriding antar Superclass dan Subclass**

File: A.java

```
package guided.gd4;
```

```java
public class A {
    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

File: B.java

```java
package guided.gd4;

public class B extends A {
    public void tampilkan() {
        super.tampilkan();
        System.out.println("Method milik class B dipanggil...");
    }

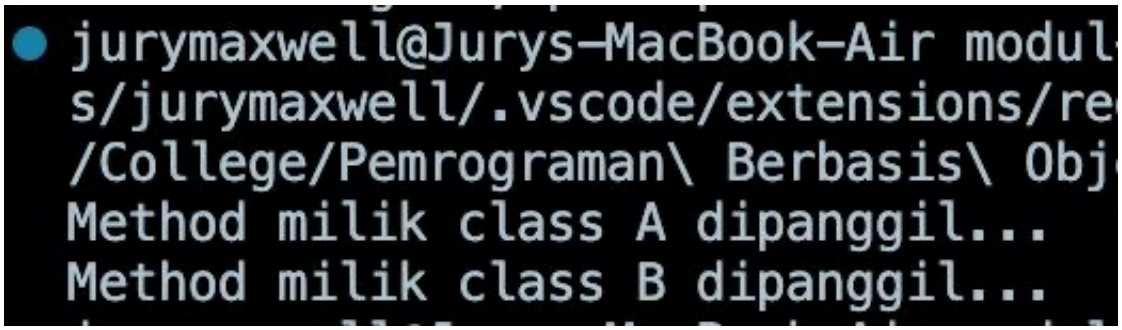
}
```

File: Main.java

```java
package guided.gd4;

public class Main {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output :



C. Unguided

1. Soal Unguided 1

Sebuah tim machine learning ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan

1. Kelas induk (model umum)

• Menyimpan nama model (teks) dan jumlah data latih (bilangan bulat).

• Memiliki constructor yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.

• Memiliki method untuk menampilkan informasi.

• Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (overloading):

• Tanpa parameter: biaya = jumlah data latih × 2.

• Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

2. Kelas turunan pertama (model regresi)

• Mewarisi kelas induk dan tambahkan atribut jumlah fitur.

• Constructor menerima nama, jumlah data latih, dan jumlah fitur; gunakan constructor kelas induk.

• Override method: tampilkan informasi dari kelas induk.

• Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).

• Tambahkan satu method baru dengan nama yang sama dengan method penghitung biaya, tetapi menerima dua parameter bilangan bulat (jumlah epoch dan ukuran batch): biaya = jumlah data latih × epoch ÷ ukuran batch.

Sehingga objek kelas ini memiliki tiga versi method penghitung biaya.

3. Kelas turunan kedua (model klasifikasi)

• Mewarisi kelas induk dan menambahkan atribut jumlah kelas.

• Constructor menerima nama, jumlah data latih, dan jumlah kelas; gunakan constructor kelas induk.

• Override method: tampilkan informasi dari kelas induk.

• Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.

• Versi penghitung biaya dengan parameter epoch tidak di-override.

4. Kelas turunan ketiga (model klasifikasi deep learning)

• Mewarisi kelas turunan kedua (bukan kelas induk langsung) dan menambahkan atribut jumlah layer.

• Constructor menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan constructor kelas di atasnya.

• Override method: tampilkan informasi dari kelas di atasnya.

• Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

5. Class Main

• Buat tiga objek berikut:

| Jenis model | Nama | Data latih | Atribut tambahan |

|---|---|---|---|

| Regresi | "Regresi Linier" | 500 | 4 fitur |

| Klasifikasi | "Decision Tree" | 800 | 3 kelas |

| Klasifikasi deep learning | "Neural Network" | 1000 | 5 kelas, 3 layer |

• Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.

• Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.

• Setelah perulangan, tampilkan total biaya.

• Terakhir, tampilkan juga:

• biaya 10 epoch untuk model "Neural Network" (ambil dari array),

• biaya 10 epoch untuk model "Regresi Linier",

• biaya 10 epoch dengan ukuran batch 50 untuk model "Regresi Linier".

Jawaban :

File: ModelML.java

```
package unguided;

public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public String getNamaModel() {
        return namaModel;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
```

```
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + this.namaModel + " | Data latih: "
+ this.jumlahDataLatih);
    }

    public int hitungBiaya() {
        return this.jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return this.jumlahDataLatih * epoch;
    }
}
```

File: ModelRegresi.java

```
package unguided;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int
jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public int getJumlahFitur() {
        return jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + this.jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int batchSize) {
        return (getJumlahDataLatih() * epoch) / batchSize;
    }
}
```

File: ModelKlasifikasi.java

```java
package unguided;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int
jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    public int getJumlahKelas() {
        return jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + this.jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * this.jumlahKelas;
    }
}
```

File: ModelDeepLearning.java

```java
package unguided;

public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public int getJumlahLayer() {
        return jumlahLayer;
    }

    @Override
```

```
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + this.jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahLayer * 200);
    }
}
```

File: Main.java

```
package unguided;

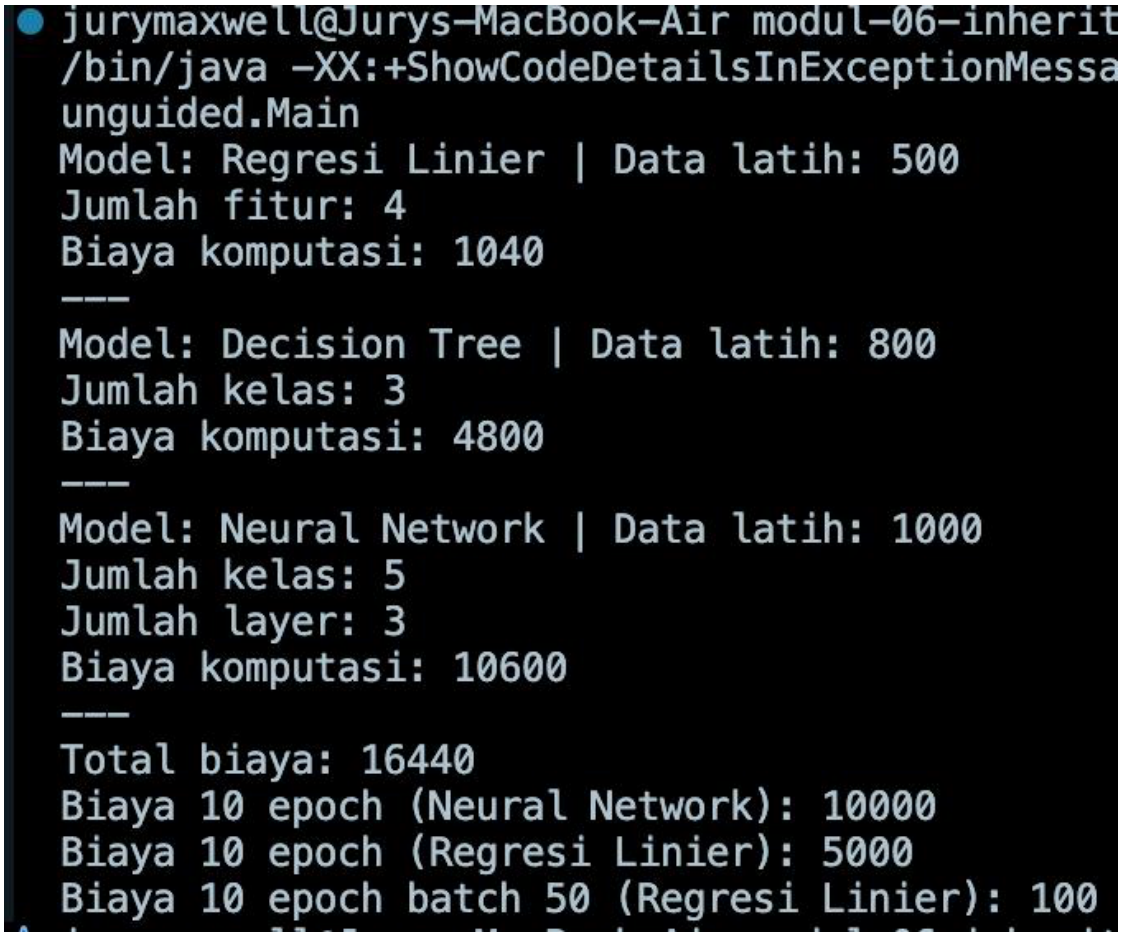
public class Main {
    public static void main(String[] args) {
        ModelRegresi regresi = new ModelRegresi("Regresi Linier", 500,
4);
        ModelKlasifikasi klasifikasi = new ModelKlasifikasi("Decision Tree",
800, 3);
        ModelDeepLearning deepLearning = new ModelDeepLearning("Neural
Network", 1000, 5, 3);

        ModelML[] models = new ModelML[] { regresi, klasifikasi,
deepLearning };

        int totalBiaya = 0;
        for (ModelML m : models) {
            m.tampilkanInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network): " +
models[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
models[0].hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
((ModelRegresi) models[0]).hitungBiaya(10, 50));
    }
}
```

Output :



Penjelasan :

Program estimasi biaya komputasi machine learning ini dikembangkan untuk memodelkan hierarki pelatihan model kecerdasan buatan dengan mengimplementasikan prinsip pewarisan bertingkat (multilevel inheritance), enkapsulasi atribut privat, serta integrasi method overloading dan method overriding. Sistem ini memfasilitasi tim perekayasa data dalam menghitung dan membandingkan beban komputasi berbagai arsitektur model secara modular, mulai dari model regresi linier sederhana, model pohon keputusan klasifikasi, hingga model jaringan saraf tiruan deep learning bertingkat banyak berdasarkan volume data latih, jumlah fitur, kelas klasifikasi, serta konfigurasi iterasi epoch dan batch size.

Pada perancangan strukturnya, kelas ModelML bertindak sebagai superclass yang mengenkapsulasi atribut privat namaModel dan jmlDataLatih. Kelas ini menyediakan method tampilkanInfo serta method hitungBiaya yang di-overload dalam dua varian, yaitu varian dasar tanpa parameter (mengalikan data latih dengan dua) dan varian berparameter epoch tunggal. Selanjutnya, kelas ModelRegresi dan ModelKlasifikasi mewarisi ModelML secara langsung. ModelRegresi menambahkan atribut spesifik

jmlFitur, meng-override method hitungBiaya dasar dengan formula penyesuaian bobot fitur, serta memperkenalkan method overloading ketiga yang menerima parameter epoch dan batch size. Di sisi lain, ModelKlasifikasi mengkhususkan diri dengan menambahkan atribut jmlKelas dan melipatgandakan biaya komputasi dasar sesuai jumlah kelas klasifikasi target.

Penerapan pewarisan bertingkat terwujud nyata pada kelas ModelDeepLearning yang mengekstensi ModelKlasifikasi, sehingga mewarisi seluruh karakteristik dari ModelML dan ModelKlasifikasi secara berjenjang. Kelas ini memperkaya kapabilitas komputasi dengan atribut jmlLayer dan meng-override method hitungBiaya untuk mengakomodasi beban pelatihan jaringan saraf berlayer banyak. Pada kelas Main, ketiga objek model diinstansiasi dengan parameter konkret dan disimpan ke dalam array polimorfik bertipe superclass ModelML. Melalui perulangan tunggal, program mengeksekusi method informasi dan biaya secara dinamis, mengkalkulasi total biaya kumulatif sebesar 16440, serta mendemonstrasikan fleksibilitas kalkulasi epoch dan mini-batch melalui explicit type casting yang aman.

D. Kesimpulan

Berdasarkan praktikum Modul 6 mengenai Inheritance, dapat disimpulkan bahwa pewarisan sifat merupakan mekanisme krusial dalam pemrograman berorientasi objek yang memungkinkan pembentukan hierarki kelas yang terstruktur, efisien, dan mudah dipelihara. Dengan memanfaatkan kata kunci extends dan super, kelas turunan dapat mewarisi atribut serta perilaku kelas induk sekaligus melakukan kustomisasi logika secara spesifik tanpa menduplikasi basis kode yang telah ada.

Selain itu, perpaduan antara method overloading dan method overriding memberikan fleksibilitas polimorfisme yang tinggi dalam penanganan komputasi. Overloading memungkinkan penyediaan berbagai antarmuka kalkulasi biaya komputasi berdasarkan kelengkapan parameter pelatihan, sedangkan overriding memastikan bahwa setiap varian model komputasi—baik regresi, klasifikasi, maupun deep learning—menjalankan logika kalkulasi yang paling tepat sesuai karakteristik arsitekturnya masing-masing.